Pramaan Rebuild Plan - 6 People

SIH26150 - Multi-Vendor DVR/NVR Forensic Analysis Tool. Internal build & assignment handout. Audited live on main @ da375dd - every menu clicked with real evidence. 6-person split.

Companion to the SIH26150 problem statement. This is the handout: read Section 1, then your section.

Audited live on main @ da375dd — every menu was clicked, every feature exercised with real data (real Dahua .dav, a real camera-card E01, a real RTSP camera). Findings below are things that broke when clicked, not guesses.

> **DESIGN GROUND RULE — read once, applies to everyone.** The app is **light theme, full stop.** The dark / "black instrumentation" UI direction was proposed and **rejected**. Ignore DESIGN.md and any note, comment, or token that says "go dark", "near-black", "navy shell", or "instrumentation theme" — that guidance is dead. Do not add dark surfaces, gradient hero banners, or text-white overrides. If you touch any screen, it stays on the light palette: white surfaces, near-black text, one blue accent. The only place dark still leaks in today is the Report HTML preview (Jai Pranav's fix) — everything else is already light and must stay that way.

1. How the App Works and Where It Stands

This section is the shared context. Read all of it before your own section — you need to know which of the eight problem-statement requirements your menu is responsible for, and what a judge sees today when they open the app cold.

1a. What SIH26150 asks for vs what exists today

The problem statement wants eight things. Three work. Five do not.

The PS asks for	State	Why
Auto-identify DVR/NVR model	MISSING	Only brand is detected, by counting magic bytes. The "confidence" is 0.42 + count*0.01 — a made-up formula; the real .dav scores 0.98 every time. model / serial / firmware are never filled.
Parse proprietary filesystem	1 of 2, unproven	The Hikvision parser is only ever run against fixtures it builds itself in the same file — the byte offsets are guesses. Dahua is a frame carver that never reads the DHFS index.
Forensic disk imaging	WORKS	Read-only chunked copy, checkpoint/resume, MD5+SHA-256.
Extract video + metadata	HALF	Byte ranges + a timestamp. No resolution, fps, codec, camera name, or event type shown anywhere.
Decode proprietary format to playable	WORKS (Dahua)	Real .dav exports to a clean MP4.
Recover deleted footage	BROKEN	See 1b Recovery. Dahua "deleted" = a 4 KB gap between carves. The generic path found 6 real deleted files in the E01 and then the recovery job crashed throwing all 6 away (byte-offset bug).
Normalise time across cameras	MISSING	temporal_sequencing.py is dead code, imported nowhere. What ships is one manual offset per device, typed in by hand, never tested with more than one channel.
Chain of custody + hashing + audit	WORKS, and is good	Append-only SHA-256 chain, evidence-digest binding, report blocked on a broken chain, signed PDF, signed case export/import. This is the strongest part of the app. Extend it, do not touch it.

The PS explicitly says: *"teams realistically support one to two brands deeply, not all eight."* We did the opposite — 8 vendor names, 5 of which (CP Plus, TP-Link, Godrej, Uniview, Matrix) are just brand strings routing to generic carving.

1b. The menu tour — what a judge sees today

The left sidebar has 13 destinations: two global (Cases, Settings) and eleven inside a case (Overview, Live devices, then the numbered workflow Acquisition -> Identification -> Recovery -> Timeline & playback -> Findings, then Custody, Report, Evidence catalog, Job log). The "Verdict" column is the disposition each menu gets in this plan: **KEEP** (works, leave it), **LIGHT FIX** (small polish), **FIX** (real bug to close), **REVAMP** (rebuild the core of the page), **TRIM** (real but overbuilt — cut it down). "What actually happens" is what I saw when I clicked it with real evidence loaded.

#	Menu	Verdict	What actually happens when you use it
—	Cases	KEEP	Works. Clean registry, new-case + import.
—	Overview	LIGHT FIX	Works, but shows a phantom "Unknown OEM // pending adapter" recovery job, and counts a camera-card E01 as "OEM identified 100%".
—	Live devices	BROKEN	The camera grid never loads. The stream URL is built as stream.mjpeg?channel=1&fps=6?t=0 — a second ? instead of & — so the server returns 422 and every tile shows "Stream unavailable". The MJPEG endpoint itself works fine; this is a one-character bug from da375dd.
1	Acquisition	FIX	"Operator drop folder" shows "No images found" even though the API returns 10 files; the detected-disk list is also blank. One failing sibling call in loadSources() wipes the whole source panel. Block-imaging is a free-text \\.\PhysicalDriveN box.
2	Identification	REVAMP	Runs. Shows "HIGH" / "MEDIUM" badges from the fake confidence formula. No model / serial / firmware. The "Layout tree" is ~15 identical rows all labelled "Dahua DHAV frame" — no hierarchy, no value. Reports a false "Filesystem / MBR" hit on a single-file .dav.
3	Recovery	REVAMP	Works for the Dahua .dav (1 segment, real 2017 timestamp). The adapter is a raw dropdown of internal keys (dahua_dhav, hikvision, h264_carve...). There is no "deleted" column or count. Running the generic adapter on the real E01: pytsk3 found 6 deleted files, then the job failed — Invalid recovered byte range 2053899776 for 5982328-byte source (filesystem offsets vs. E01-file offsets mismatch). The 2 rows that survived are a 1-byte file and a 16 KB blob from parser "unknown".
4	Timeline & playback	REVAMP	Timeline bar renders for the .dav. The playback deck shows a blank white lane — the ranged export (-ss before -i + -c copy, the round-5 "speed fix") produces a 261-byte MP4 with zero streams, so the <video> errors (code 4) with no message to the user. Only ever one channel, so cross-camera alignment can't be shown.
5	Findings	TRIM	Real detectors run (OpenCV MOG2, YuNet, YOLOX). Output on a CCTV still: "clock candidate", "wine glass candidate", "bowl candidate" at 0.35–0.62 — junk. "Scene change" pipeline never fires. Code still says demo_mode_unavailable.
—	Custody	KEEP	Works well. 8 events, hash-linked, "INTACT". Per-entry hash isn't shown in the table — worth adding.
—	Report	FIX	Live HTML preview + signed PDF, real content, honest disclaimer. But the preview renders dark-themed inside a light app, and "Recovered sequences (2 jobs) / Segments 2" is inflated by the phantom job.
—	Evidence catalog	LIGHT FIX	Works. Still has a dead "Lab specimen" filter. 00-style leading-zero counts look fake. Inspector doesn't show a recovered-recording count.
—	Job log	FIX	Shows only recovery/imaging jobs — analytics, live-capture and export jobs are invisible. Phantom blank-vendor recovery job present. No job-kind column.
—	Settings	FIX	Data dir + signing fingerprint + dataset fetch are fine. "Parser sanity check -> Run" is a pytest regression harness shipped as a product feature.

1c. Remove entirely — fake / lab / demo / dead

Delete	Where
Synthetic-specimen acquisition (writes fabricated evidence into a real case)	POST /devices/acquire/synthetic, source=synthetic_specimen branch in api/v1/devices.py, create_lab_specimen in services/acquisition.py, verification/lab_specimen.py + hikvision_specimen.py + honeywell_specimen.py, api.acquireSynthetic in src/lib/api.ts
"Lab specimen" evidence category	CaseEvidenceCatalogPage.tsx (flask icon, inferCategory branch, facet option)
"Parser sanity check" as a product feature	SettingsPage.tsx panel, api/v1/tool_verification.py, services/tool_verification_report.py, verification/run_suite.py — move the checks to pytest/CI
Dead code	parsers/temporal_sequencing.py (imported nowhere); schemas/dhav.py seal_dhav_frame / build_dhav_timestamp_tlv / parse_dhav_timestamp_tlv (NotImplementedError stubs); b"HKVI" magic in filesystem_recovery.py (invented)
8-vendor theatre	Collapse manufacturer_detect.py OEM_PROFILES to Dahua + Hikvision + one honest "generic filesystem / H.264 recovery — no proprietary parsing" line. Drop CP Plus / TP-Link / Godrej / Uniview / Matrix as separate profiles.
Every fabricated confidence number	identify_image (0.42 + count*0.01), recovery._confidence_score (0.92/0.7/0.55), DhavValidationResult.confidence (0.95/0.72/0.45), _confidence_label, frontend tierOf. Replace with the list of checks that passed + one word: strong / weak / none.
"demo" language	rename demo_mode_unavailable to analytics_unavailable (engine + UI)

1d. The one thing that blocks everyone

There is no real DVR **disk image** with a **deleted** recording anywhere in the repo. The .dav is a single-file export — no filesystem, nothing deleted. Every parser is validated only against self-built fixtures. **Before any parser work starts**, someone must obtain a real Hikvision or Dahua disk dump, or build one faithfully from published reverse-engineering (real FS layout, real recordings written, one recording deleted through the recorder, then imaged). If neither can be done, the fallback is an emulated image and the report must state plainly which parts are emulated. This is priority zero.

2. Team Structure

Person	Menus owned	Engine / code owned	Weight
Navneet	Identification (REVAMP), Acquisition (FIX)	manufacturer_detect.py, device_structure.py, services/acquisition.py, services/disk_enumeration.py, services/e01_reader.py	Medium-heavy
Aravind	(engine behind Recovery)	parsers/hikvision.py, parsers/schemas/hikvision_fs.py, the real Hikvision disk image, test_e2e_hikvision.py	Heavy (the proprietary-FS reverse-engineering)
Aslam	(engine behind Recovery + Timeline playback)	parsers/dahua_dhfs.py, parsers/schemas/dhav.py, services/recovery.py artifact writing, sequences/{id}/export endpoint, MPEG-PS unwrap, playable_frame_count	Heavy
Roshan	Recovery (page REVAMP), Evidence catalog (LIGHT FIX)	CaseRecoverPage.tsx, SegmentInspector.tsx, parsers/generic_tier2.py, parsers/filesystem_recovery.py, parsers/generic_fallback.py	Heavy
Soumika	Timeline & playback (REVAMP), Findings (TRIM)	services/timeline.py, parsers/temporal_sequencing.py, CaseTimelinePage.tsx, PlaybackDeck.tsx, TimelineView.tsx, services/ai_analytics.py, CaseAiAnalyticsPage.tsx	Heavy

Person	Menus owned	Engine / code owned	Weight
Jai Pranav	Live devices (FIX), Report (FIX), Custody (KEEP), Overview (LIGHT), Cases (KEEP), Job log (FIX), Settings (FIX)	services/live_devices.py, api/v1/live.py, CaseLiveDevicesPage.tsx, services/logical_acquisition.py, services/reporting.py, scripts/dev/rtsp_test_source.py, Tauri shell + CI + the MVP demo script + the Section 1c purge	Medium + breadth + integration

Numbers 2, 3, 5 are deep single-focus. 4 is a page revamp plus one engine path plus a small menu. 6 is one real chunk (Live devices) plus six light menus plus the glue role. Roughly even.

3. Per Person

3.1 NAVNEET — Identification + Acquisition

Note for this area: you own the first two things a judge touches. Both currently show fabricated confidence numbers and neither surfaces a model. Nothing you build blocks others *except* that Aravind and Aslam need the device-info byte offsets from your identification research — pair with them early on where the model/serial string lives in each vendor's layout.

Menu 2 — Identification (REVAMP)

- Current: manufacturer_detect.identify_image counts brand magic bytes, emits 0.42 + count*0.01 as "confidence", a flat repetitive "layout tree" of raw magic hits, no model.
- Remove: the confidence formula; the 8-vendor OEM_PROFILES (keep Dahua + Hikvision + one generic line); the synthetic-acquire code path end to end (see 1c).
- Fix: dedupe the "Filesystem" hits (one card, not one per marker); suppress the false "MBR" hit on non-disk single files.
- Build: read the recorder's device-info region and return **make, model, firmware, disk serial, channel count** — render them as a device card at the top of the page. Each hit returns an evidence object: {signatures_found, byte_offsets, fs_layout_parsed, strength: strong|weak|none}. The layout tree shows the parsed FS structure (partition table, index region, video area) — not a list of frame hits.
- Done when: dropping the real Hikvision and Dahua images shows the correct make/model/firmware/serial/channel-count with zero percentages, and an evidence list with real byte offsets.

Menu 1 — Acquisition (FIX)

- Current: "Operator drop folder" and the disk list both render blank despite working APIs (one failing call in loadSources() aborts the chain and the catch clears everything). Block-imaging is a raw \\.\PhysicalDriveN text box.
- Fix: make each source load independent (Promise.allSettled, not a single try/catch) so a failing disk-enumeration call doesn't hide the drop folder. Add .dav and .mp4 to the file-input accept.
- Build: make the detected-disk list the primary imaging control — click a disk, it images. Keep the typed path behind an "Advanced" toggle.
- Done when: with validation_data/oem/ populated, the drop folder lists every file and the disk list shows every detected disk, even if one of the three source calls fails.

Engine you own: manufacturer_detect.py, device_structure.py, services/acquisition.py, services/disk_enumeration.py, services/e01_reader.py.

3.2 ARAVIND — Recovery engine: Hikvision proprietary filesystem

Note for this area: this is the hardest single task and the demo depends on it. You do not touch the Recovery page (Roshan) — you produce a clean list of recording entries; everything downstream reads that list, not the disk.

- Priority zero, with Jai Pranav: get or build the real Hikvision disk image (see 1d). Then write docs/reference/hikvision_fs.md — every byte offset you rely on, each with a public source (HIKBTREE research, DVR Examiner notes, the MDPI 2025 Hikvision/Dahua forensic paper).
- Current: schemas/hikvision_fs.py parses a master block at 0x200 and walks HIKBTREE, but only against fixtures it builds itself (build_master_block, build_hikbtree_page). Offsets 0x98, 0x58, the 48-byte entry, flag == 0 — all

unverified.
- Fix: validate/correct every offset against the real image. Move the build_* helpers to a test-only module. Confirm _scan_mapped never does bytes(data) on the whole mmap.
- Build: **deleted classification** — a HIKBTREE entry whose allocation flag is cleared but whose data_offset still points into the video area is a deleted recording -> mark it deleted (index entry cleared) with a lowered timestamp_confidence. Extract event type + resolution + fps per entry from the PS stream headers.
- Output shape (Roshan and Soumika consume this): channel, start_ts, end_ts, byte_offset, byte_length, event_type, resolution, fps, allocation_state.
- Done when: parse_hikbtree_entries(real_image) returns the exact recording count you counted by hand, at least one marked deleted, and a test asserts both the deleted one and the exact count.

Engine you own: parsers/hikvision.py, parsers/schemas/hikvision_fs.py, the Hikvision sample image, test_e2e_hikvision.py.

3.3 ASLAM — Recovery engine: Dahua + the carve-to-playable pipeline

Note for this area: two jobs — turn Dahua from a frame carver into a filesystem parser, and own the pipeline that turns any recovered byte range (Hikvision or Dahua) into an MP4 that actually plays. The playback deck is broken right now because that pipeline produces empty files (see below) — fixing that is on you.

Dahua filesystem

- Current: dahua_dhfs.py scans for DHAV magic and stitches contiguous frames; never reads the DHFS index. DhavValidationResult.confidence = 0.95/0.72/0.45.
- Build: locate and parse the DHFS index / allocation structure (partition marker DHFS4.1). Enumerate recordings from the index. Keep the DHAV magic scan as an explicit fallback labelled carve_fallback for damaged indexes. Frames whose byte range is outside any allocated recording -> deleted (unallocated extent).
- Replace confidence with a passed-checks list.
- docs/reference/dhfs.md with sources. Use a real Dahua disk image if one can be obtained (second priority after Hikvision).

Carve-to-playable pipeline (both vendors)

- Current bug (found in this audit): the Timeline ranged export runs ffmpeg -ss <t> -i <carved .h264> -c copy -f mp4 and produces a **261-byte MP4 with no streams** — -ss before -i on a raw H.264 elementary stream (no container index) seeks to nothing, and -c copy writes zero frames. The playback <video> then fails silently.
- Fix: for raw-H.264 artifacts, -ss must come **after** -i, or the artifact must first be wrapped in a container with timestamps. Decode-and-re-encode the window (-c:v libx264 -preset ultrafast) if copy can't produce a valid head. The full-segment export path (which does work) and the ranged path must both yield a file ffmpeg -f null - reads clean.
- Verify the MPEG-PS / H.264 unwrap in schemas/hikvision_fs.py:wrap_mpegps against a real Hikvision stream.
- playable_frame_count: compute it on export for both vendors, matching ffprobe -count_frames. Stop emitting frame_count as if it were a frame count; keep container_units (raw packs/frames) + playable_frame_count.
- Done when: every recovered recording — allocated or deleted, Hikvision or Dahua, full or 30-second window — exports to an MP4 that decodes clean and reports the right frame count, and the Timeline playback deck plays it.

Engine you own: parsers/dahua_dhfs.py, parsers/schemas/dhav.py, services/recovery.py (artifact writing), api/v1/devices.py export endpoints, test_dahua_real_dav.py.

3.4 ROSHAN — Recovery menu (page) + generic/E01 recovery + Evidence catalog

Note for this area: you own the Recovery page and the generic filesystem path. The generic path is the one that is *supposed* to be reliable and it currently crashes on a real E01 after finding 6 real deleted files — fixing that is the centre of your work.

Generic / E01 recovery — the crash

- Current bug (found in this audit): generic_tier2 -> filesystem_recovery.py (pytsk3) found **6 deleted files** in nps-2013-canon1.E01, then recovery.py:_write_bounded_artifact threw ValueError: Invalid recovered byte range

[2053899776, 2053900288) for 5982328-byte source and the whole job **failed**. The filesystem is reporting offsets into the full logical volume; the artifact writer is validating them against the E01 container file size. The two coincidental survivors are a 1-byte file and a 16 KB blob from parser "unknown".
- Fix: read recovered files through the same filesystem/EWF handle that found them (TSK file-object read), not by seeking raw byte offsets into the E01 file. A recovered file's bytes come from tsk_file.read_random(...), not open(e01).seek(offset).
- Remove the b"HKVI" magic from filesystem_recovery.py.
- Make the result and the UI say plainly: "generic filesystem recovery — not proprietary DVR parsing".
- Done when: generic_tier2 on both nps-2013-canon1.E01 and nps-2009-canon2-gen6.E01 completes (no crash) and recovers the documented deleted files with an exact count; every recovered file has real bytes, a real size, and a real parser name.

Menu 3 — Recovery page (REVAMP)

- Current: raw adapter-key dropdown; the dropdown doesn't follow the evidence selection; a "Confidence tiers" donut built from the fake numbers; no deleted column.
- Fix: adapter is chosen automatically from Identification; the manual dropdown goes behind an "Advanced" toggle; the dropdown updates when you change evidence.
- Build: a **Deleted** column, a header count ("4 recordings, 1 deleted"), a "deleted only" filter, a distinct row style for deleted rows — fed from the parsers' real allocation_state (Aravind / Aslam), not a validation-string guess. Replace the confidence donut with a checks-passed breakdown. The SegmentInspector shows real metadata: channel, start/end, resolution, fps, codec, event type, both hashes.
- Done when: recovering the real Hikvision image shows N recordings with a correct deleted count and a filter that isolates them.

Menu — Evidence catalog (LIGHT FIX)

- Remove the "Lab specimen" category, flask icon, facet option, and the filename.includes("specimen") logic.
- Category from acquisition_method / media_type; status from verification_status; every facet option shows a real count (no 00 padding).
- Add a recovered-recording count to the inspector panel.

Engine you own: CaseRecoverPage.tsx, SegmentInspector.tsx, parsers/generic_tier2.py, parsers/filesystem_recovery.py, parsers/generic_fallback.py, CaseEvidenceCatalogPage.tsx.

3.5 SOUMIKA — Timeline & playback + Findings

Note for this area: the Timeline is where the case comes together — multi-camera, normalised time, deleted segments visible — and it's the PS requirement nobody has built. The playback deck is broken (Aslam owns the export bug; you own everything above it). Findings is real but noisy; trim it.

Menu 4 — Timeline & playback (REVAMP)

- Current: pick a device; "Clock drift calibration" = one wall-Unix + one device-Unix -> one global offset for that device; a per-channel bar; a "MOTION / AI FINDINGS" track; a multi-lane deck where each lane seeks independently and currently renders blank. timeline.py is a pure DB projection; deleted_candidate is guessed from a validation-string set.
- Build:
  - a **2-channel test case** from existing real clips (two files as two channels) so you can work before the parsers deliver deleted flags;
  - a per-channel offset stored **per channel**, not one number per device;
  - a **shared absolute time axis** in timeline.py + TimelineView.tsx when >=2 channels have recorder timestamps;
  - per-lane offset nudge UI + auto-estimate from overlapping motion events (or recorder NTP markers if present);
  - one transport in PlaybackDeck.tsx — one scrub seeks **every** lane to the same absolute instant (currently independent);
  - the deleted lane in a distinct colour, with a legend and a "deleted only" toggle, fed from the parsers' real allocation_state.

- Wire real deleted state through timeline.py instead of the string-set guess.
- Delete temporal_sequencing.py, or make sequence_frames real and wire it in with a test.
- Done when: the 2-channel case shows both lanes on one absolute axis, per-lane offset adjustable, one scrub moving both, and deleted segments visually distinct.

Menu 5 — Findings (TRIM)

- Rename demo_mode_unavailable -> analytics_unavailable (engine + UI).
- Motion noise floor: drop confidence < 0.35, or keep the top 8 per sequence, or merge motion events < 2 s apart into one span. Document which in the finding's bbox_json.
- Keep the "Include in report" toggle. If the team decides a separate menu isn't worth it, fold Findings into the Timeline "MOTION / AI FINDINGS" track and drop menu 5 — decide with Jai Pranav.
- Done when: on the real .dav, findings <= 10 and every one is above the floor.

Engine you own: services/timeline.py, parsers/temporal_sequencing.py, CaseTimelinePage.tsx, PlaybackDeck.tsx, TimelineView.tsx, FindingsTrack.tsx, services/ai_analytics.py, CaseAiAnalyticsPage.tsx.

3.6 JAI PRANAV — Live devices + Report + Custody/Overview/Cases/Job log/Settings + integration

Note for this area: one real feature (Live devices, currently broken by a one-character bug), six menus that are mostly fine and need trimming, and the glue role — you run the fake-feature purge, own the weekly MVP demo, and keep custody/report rock-solid.

Menu — Live devices (FIX)

- Current bug (found in this audit): the grid URL is stream.mjpeg?channel=1&fps=6?t=0 — a cache-buster ?t= appended to a URL that already has a query string -> **422**, every tile "Stream unavailable". CaseLiveDevicesPage.tsx line ~47.
- Fix: build the URL properly (&t= or a URL object). Then re-verify against a real RTSP source with G.711 audio that the grid shows live video, "Expand" plays with audio, and Snapshot / Capture land in custody with correct hashes.
- Build: the grid tiles **all** connected devices' channels (devices.flatMap(channels)), not just the selected one. Wire the "Open pull panel" — channel <select> + start/end datetime-local -> logical_acquisition bounds into ISAPI CMSearchDescription / Dahua condition.*. Exercise the ONVIF path against a simulator or a real camera.
- Fix scripts/dev/rtsp_test_source.py — it crashes on Windows (missing default file, a -> character the console can't print). Real default source, ASCII only, publish 3 paths.
- Move test_live_devices_integration.py into the default CI run (it's -m integration and skipped now); the mocked test_live_devices.py does not count as proof.
- Done when: two cameras added -> grid shows both live -> Expand plays with audio -> snapshot + 20 s capture in custody/evidence with correct hashes -> one recording pulled by channel + time.

Menu — Report (FIX)

- Current: real content, honest disclaimer, signed PDF, blocked on broken chain — all good. But the HTML preview renders **dark** inside a light app, and "Recovered sequences (2 jobs) / Segments 2" is inflated by a phantom job.
- Fix: **restyle reporting.py's generated HTML to the light theme** — white background, near-black text, one blue accent, matching every other screen. This is the last place the rejected dark UI still shows; kill it (see the Design Ground Rule at the top of this doc — DESIGN.md is dead, no dark surfaces anywhere). Count real recovered recordings, not job rows.
- Build: add a device block (make / model / firmware / serial / channel count from Navneet), a **deleted-recovery count** section, and a per-recording metadata table (from Aravind / Aslam / Roshan). Confirm report_state=INCLUDED findings render in the PDF, not just JSON/HTML.

Menu — Job log (FIX): add a job-kind column (imaging / recovery / export / analytics / live-capture); show all kinds, not just recovery; failed jobs show the real error inline; kill the phantom blank-vendor recovery job at its source.

Menu — Settings (FIX): delete the "Parser sanity check" panel and its backend (tool_verification.py, tool_verification_report.py, run_suite.py). Keep data dir + fingerprint + dataset fetch.

Menu — Evidence-catalog / Custody / Overview / Cases: Custody and Cases are KEEP — add a per-entry hash column to the custody table and a multi-device custody test. Overview: surface model/firmware when identification

returns it; make "OEM identified %" reflect the strength word, not the fake confidence; kill the phantom recovery-job card.

Cross-cutting — you own:

- Execute the entire Section 1c purge. Sign-off: grep -rniE "synthetic|specimen|tool.?verification|temporal_sequencing|acquireSynthetic|HKVI|demo_mode" src engine/app returns only real renamed usages.
- With Aravind: land the real disk image (1d).
- Write scripts/demo/mvp.sh: create case -> acquire real image -> identify (model shown) -> recover (deleted segment found) -> timeline (2 channels normalised) -> custody log -> signed report. It fails at first — that failure list is the team's backlog. Run it on every merge; block anything that regresses it.
- Own the Tauri desktop shell + CI config.

Engine you own: services/live_devices.py, api/v1/live.py, CaseLiveDevicesPage.tsx, services/logical_acquisition.py, services/reporting.py, CaseReportPage.tsx, CaseJobsPage.tsx, SettingsPage.tsx, CaseCustodyPage.tsx, CaseOverviewPage.tsx, CasesPage.tsx, scripts/dev/rtsp_test_source.py, scripts/demo/mvp.sh.

4. Continuity Map — the MVP demo, end to end

This is the run a judge does. Every row must work on the real image. mvp.sh (Jai Pranav) automates it and is the acceptance gate.

Step	Menu	Owner	What must be true
1. Create case	Cases	Jai Pranav	Case + custody chain start. Already works.
2. Acquire the disk image	Acquisition	Navneet	Drop-folder lists the real image; register it; hash verified.
3. Identify	Identification	Navneet	Correct make + model + firmware + serial + channel count. No percentages. FS layout tree.
4. Recover	Recovery	Roshan (page) + Aravind / Aslam (engine)	N recordings listed with real metadata; at least one marked DELETED; a "deleted only" filter isolates it. Job does not crash.
5. Play a recovered recording	Recovery / Timeline	Aslam (pipeline) + Soumika (deck)	The recording — including the deleted one — exports to an MP4 that plays, with the right frame count.
6. Normalised timeline	Timeline & playback	Soumika	>=2 channels on one absolute axis; per-lane offset; one scrub moves all lanes; deleted lane distinct.
7. Custody log	Custody	Jai Pranav	Every step above appears as a hash-linked entry; chain "INTACT". Already works.
8. Signed report	Report	Jai Pranav	PDF contains the model, the deleted-recovery count, per-recording metadata, and the custody tip hash. Blocked if the chain is broken.
(parallel) Live device	Live devices	Jai Pranav	Add a camera by IP -> grid shows it live -> snapshot into custody. Independent of steps 2–8.

5. Pre-Flight Checklist Per Person

- [ ] Read Section 1 in full — know which of the 8 PS requirements your menu is responsible for.
- [ ] Confirm nothing you touch reintroduces the rejected dark UI — light palette only, DESIGN.md ignored (see the Design Ground Rule at the top).

- [ ] Delete every fake/lab/demo item in your area from the Section 1c table, with a test that the surface is gone.
- [ ] Replace every fabricated confidence number in your code with a passed-checks list + strong/weak/none.
- [ ] npm run build + npx tsc noEmit + npx prettier check src clean on your changes.
- [ ] python -m pytest engine/tests -q green — no x, no F.
- [ ] Your menu works, clicked cold, on the real image — not on a fixture, not on your second try.
- [ ] Your "done when" line in Section 3 is satisfied, demonstrated to Jai Pranav.
- [ ] mvp.sh gets at least one step further because of your work.
- [ ] Hand your section to Jai Pranav for the integration check before it's called done.